likely want to consider minimal APIs for the performance improvements. For server-to-server APIs or nonbrowser clients, an alternative protocol like gRPC (https://docs.microsoft.com/aspnet/core/grpc) may be a good fit. You might also consider protocols like GraphQL, as discussed in *Building Web APIs in ASP.NET Core*, by Valerio De Sanctis (Manning, 2023).

Alternatively, if you're building an app with real-time functionality, you'll probably want to consider using WebSockets instead of traditional HTTP requests. ASP.NET Core SignalR can be used to add real-time functionality to your app by providing an abstraction over WebSockets. SignalR also provides simple transport fallbacks and a remote procedure call (RPC) app model. For details, see the documentation at https://docs.microsoft.com/aspnet/core/signalr.

Another option available in ASP.NET Core 7 is Blazor. This framework allows you to build interactive client-side web applications by using the WebAssembly standard to run .NET code directly in your browser or by using a stateful model with SignalR. See *Blazor in Action*, by Chris Sainty (Manning, 2022), for more details.

I hope that by this point you're sold on Razor Pages and their overall design using the MVC pattern. Nevertheless, using MVC controllers makes sense in some situations, so it's worth bearing that in mind. Another important point to remember is that you can include both MVC controllers and Razor Pages in the same application if you need them.

You've learned about MVC controllers as an alternative to Razor Pages, and in part 1 of this book you learned about using minimal APIs to build JSON APIs. Web API controllers sit somewhere in between; they use MVC controllers but generate JSON and other machine-friendly format data, not HTML. In chapter 20 you'll learn why you might choose to use web API controllers over minimal APIs and how to build a web API application.

Summary